

Dokumentacja urządzenia oraz oprogramowania wbudowanego

Opis ogólny

Powstające urządzenie ma za zadanie dokonywać odczytów z czujników jachtowych i wbudowanych, udostępniać odczyty oraz ich filtrowany trend za pomocą API REST w sieci WiFi dostępnej na jachcie lub generowanej przez urządzenie. Urządzenie ma też za zadanie udostępniać oprogramowanie klienckie, które będzie dokonywało odczytów głosowych wybranych wartości osobie sterującej jachtem w sposób nie wymagający uprzedniej instalacji oprogramowania na urządzeniu dostępowym (smartfonie, tablecie lub laptopie).

Komponenty urządzenia

Zasilanie

Urządzenia zawiera układ zasilający przyłączany do sieci NMEA jachtu oraz akumulator, który jest wykorzystywany w przypadku, kiedy nie ma możliwości przyłączenia do sieci jachtowej. Urządzenie pozwala na ładowanie akumulatora za pomocą wbudowanego portu micro USB także w czasie połączenia urządzenia z siecią jachtową. Urządzenie nie pozwala na ładowanie akumulatora bezpośrednio z sieci jachtowej z powodu możliwości jej uszkodzenia. Układ zasilania jest zabezpieczony bezpiecznikiem polimerowym wielokrotnego użytku, który chroni sieć jachtową oraz układ zasilania w przypadku awarii urządzenia. Akumulator jest podłączony do układu TP4056, który zabezpiecza go przed krytycznym rozładowaniem oraz zbyt wysokim prądem ładowania. Układ zasilania używa przetwornicy step-up/step-down co pozwala na pracę urządzenia przy napięciach sieci dochodzących do 16V oraz napięciu baterii od 3V. Oba źródła zasilania są od siebie odcięte zestawem diod Schottky'ego.

Przyłącze NMEA 2000

Urządzenie dokonuje odczytu danych z sensorów jachtu dzięki przyłączeniu do sieci jachtowej w standardzie NMEA2000 opartym o magistralę CAN. Obecnie jest to najbardziej powszechne, nowoczesne rozwiązanie stosowane na jachtach. Podłączenie urządzenia, w najtrudniejszym przypadku, wymaga dołączenia do istniejącej sieci trójnika zakończonych kablem z urządzeniem. W bardziej optymistycznym przypadku - podłączenia urządzenia do dostępnego wolnego kabla sieci jachtowej. Urządzenie udostępnia standardową końcówkę typu mini, która pasuje do kabli dostępnych w komercyjnej sprzedaży. Po stronie urządzenia za integrację z interfejsem CAN odpowiada moduł SN65HVD230. Wyjście z modułu jest przekazywane do wejścia Serial2 RX mikrokontrolera. Odczytywanie danych po stronie oprogramowania wbudowanego obsługuje klasa NMEA korzystająca z biblioteki open source [ttlappalainen/NMEA2000](https://github.com/ttlappalainen/NMEA2000). Dane odczytywane na poziomie oprogramowania to kurs drogi nad dnem, kurs rzeczywisty, prędkość nad dnem, kąt wychylenia steru, pozycja, kierunek i prędkość wiatru, czas oraz ciśnienie, temperatura i wilgotność. Kluczowymi danymi dla poprawnej pracy urządzenia są kurs drogi nad dnem i kurs kompasowy. Jeśli dane kluczowe nie są dostępne na magistrali NMEA urządzenie spróbuje odczytać je z wbudowanego sensora GPS. Pozostałe dane są udostępniane przez urządzenie w dodatkowym adresie API umożliwiając skorzystania z nich w przyszłości.

Sensor GPS

W przypadku braku dostępu do sieci jachtowej rolę kluczowego sensora kierunku przejmuje moduł GPS (w prototypie wykorzystano UBX-G7020-KT). Komunikacja z modułem GPS odbywa się za pomocą magistrali Serial1 mikrokontrolera w protokole NMEA0183. Odczytywanie danych po stronie oprogramowania wbudowanego obsługuje klasa GPS korzystająca z biblioteki [slashdevin/NeoGPS](https://github.com/slashdevin/NeoGPS). Dane odczytywane na poziomie oprogramowania to kurs drogi nad dnem, prędkość nad dnem, pozycja i czas.

Sensor IMU

W przypadku korzystania z niskiej jakości sygnału GPS przewidziano możliwość skorzystania z dodatkowego filtrowania odczytów za pomocą połączenia żyroskopu i akcelerometru (w prototypie MPU9255). Komunikacja z modułem odbywa się za pomocą magistrali I2C mikrokontrolera. Odczyt danych po stronie oprogramowania obsługuje klasa IMU korzystająca z biblioteki open source [hideakitai/MPU9250](https://github.com/hideakitai/MPU9250). Dane odczytane z sensora są filtrowane [filtrem Madgwicka](#) zbudowanym na podstawie kodu źródłowego twórcy udostępnionego w internecie. Oryginalne rozwiązanie przystosowano do pracy na systemie urządzenia, który wymusił pewne uproszczenia wynikające z obsługiwanej wersji C++ oraz możliwości sprzętowych mikrokontrolera. Wynik pracy filtra jest wykorzystywany w formie różnicy obrotu w osi pionowej aplikowanej do poprzedniego odczytu kursu nad dnem wraz z poprzednią poprawką filtra. W przypadku stabilnego kursu jachtu filtr będzie zbiegał do kursu nad dnem, aby nie dopuścić do wpływu dryfu żyroskopu na wynikowe wskazania.

Mikrokontroler

Główną jednostką przetwarzania urządzenia jest mikrokontroler ESP-32 (w prototypie esp-32 devkit C). Mikrokontroler posiada dwa rdzenie o taktowaniu 240 MHz oraz 4 MB pamięci flash. Pozwala on na łączenie się z sieciami WiFi oraz utworzenie własnego punktu dostępowego udostępniającego własną sieć WiFi niskiej przepustowości dla maksymalnie czterech urządzeń. Posiada magistrale CAN, I2C, SPI oraz wiele magistral Serial. Powyższe możliwości czynią go idealnym kandydatem do zastosowania w urządzeniu ponieważ dysponuje wymaganymi magistralami oraz interfejsem WiFi jednocześnie charakteryzując się wystarczającą pamięcią i zegarem procesora oraz niskim zużyciem energii. Ważnym czynnikiem jest też możliwość prowadzenia operacji na dwóch rdzeniach i system operacyjny pozwalający na kierowanie tym działaniem (freeRTOS), dzięki czemu można przeprowadzać operacje filtrowania bez zakłócania komunikacji z urządzeniami klienckimi.

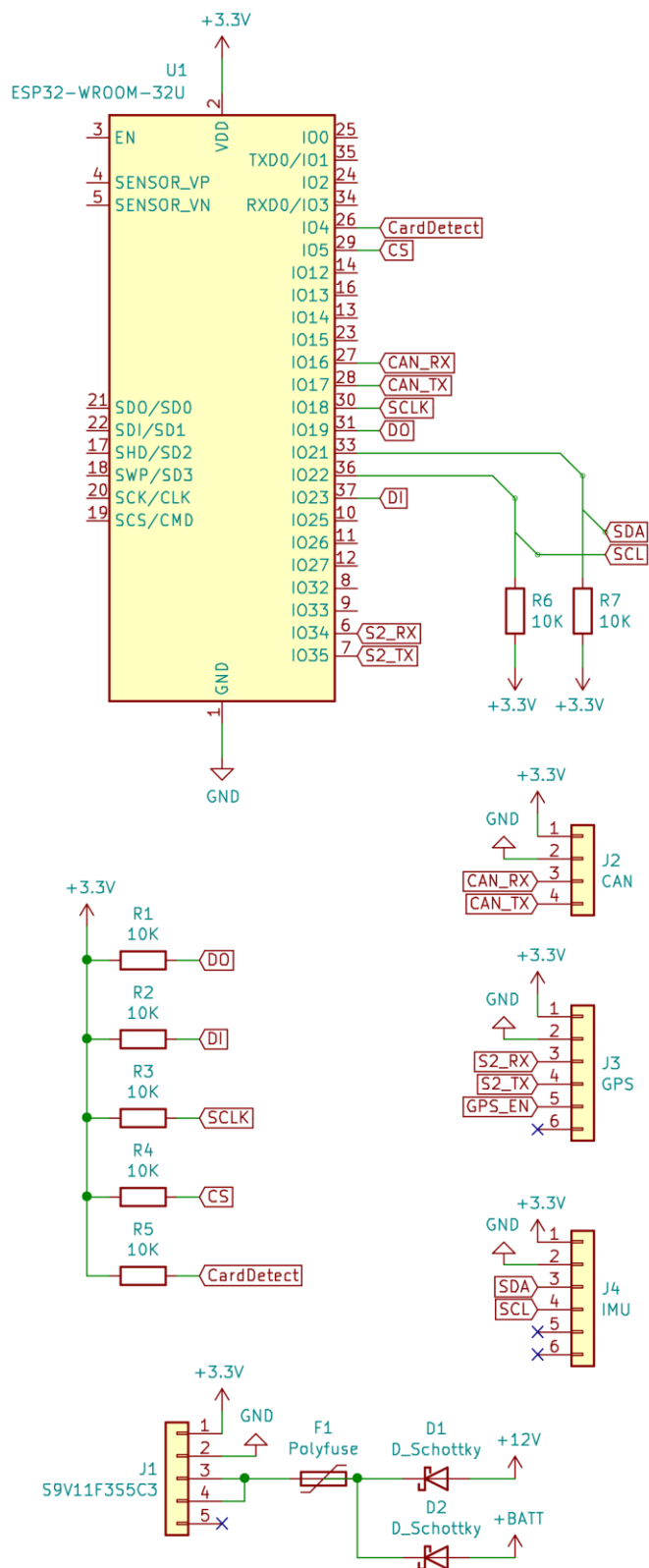
Za interfejs komunikacji z urządzeniami klienckimi po stronie oprogramowania wbudowanego odpowiada klasa RestApi korzystająca z biblioteki open source [me-no-dev/ESPAsyncWebServer](https://github.com/no-dev/ESPAsyncWebServer). Do obsługiwanego połączenia z siecią WiFi oraz

własnego punktu dostępowego używana jest klasa WiFIManager zbudowana w oparciu o natywne biblioteki ekosystemu Arduino użytego w projekcie. Aplikacja kliencka jest udostępniana przy użyciu klasy RestApi prosto z pamięci flash urządzenia przy użyciu SPIFFS. Zbudowana forma aplikacji jest przechowywana w SPIFFS we wstępnie skompresowanej formie (.gz) aby zaoszczędzić miejsce w pamięci urządzenia.

Karta SD

W celu zebrania odczytów urządzenia oraz wyników pracy filtrów i późniejszego ich opracowania do urządzenia dołączono gniazdo kart SD. Gniazdo jest podłączone do mikrokontrolera za pośrednictwem magistrali SPI. Udostępnia też informację o umiejscowieniu karty w slotcie za pomocą jednego z pinów GPIO mikrokontrolera. Na poziomie oprogramowania wbudowanego za dostęp do karty SD odpowiada klasa SDManager opierająca się o natywne biblioteki ekosystemu Arduino. Urządzenie może działać bez dostępu do karty SD, nie będzie jednak w stanie zapisywać odczytów ponieważ jego pamięć flash dostępna poprzez SPIFFS nie ma wystarczającej pojemności aby przechowywać dodatkowe dane.

Rysunek 1: Uproszczony schemat budowy urządzenia



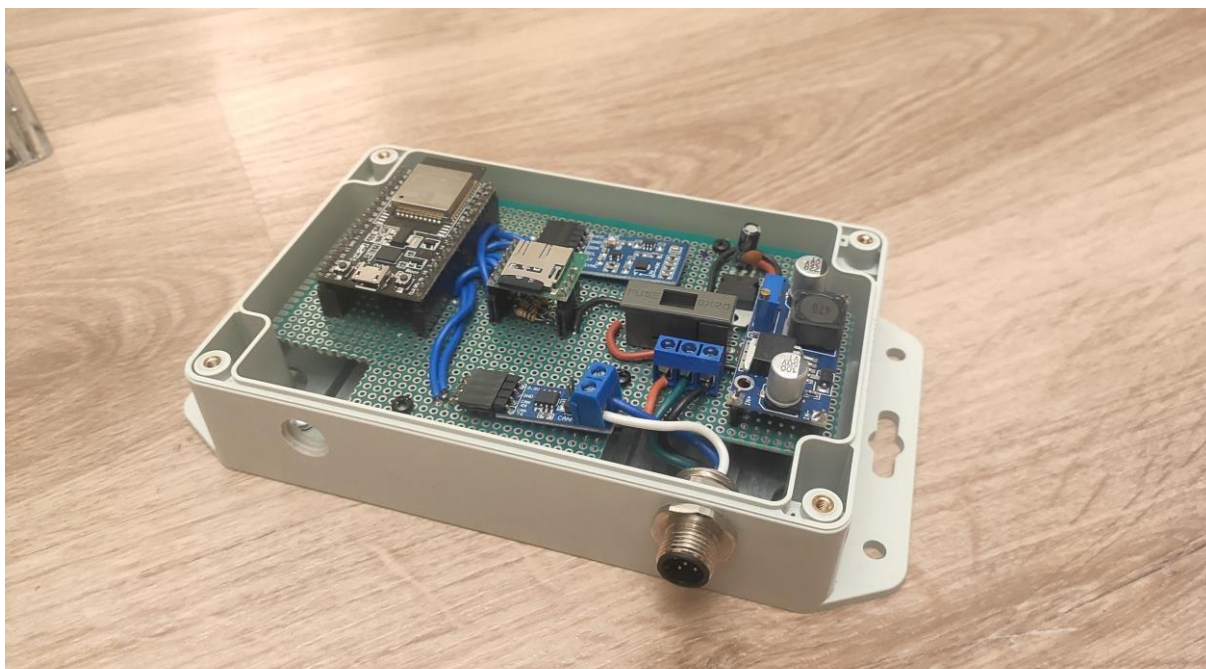
Zestaw testowy

W skład zestawu testowego wchodzi prototyp urządzenia oraz szkieletowa magistrala NMEA2000 wraz z sensorami. Do magistrali dołączono wtyczkę pozwalającą na podłączenie zasilacza sieciowego, samochodowego lub innego źródła (w tym akumulatora jachtu). Sensory to wzorcowy sensor kursu kompasowego ze wsparciem żyroskopowym [Aimar H2183](#) oraz sensor [GPS 19x NMEA2000](#) firmy Garmin. Sensor kursu kompasowego dobrano tak, aby był możliwie najlepszym porównaniem jakościowym dla pozostałych metod ustalania kierunku oferowanych przez urządzenie, ale jego odpowiednik może być też używany przy normalnej eksploatacji urządzenia.

Rysunek 2: Zestaw testowy z podłączonym prototypem



Rysunek 3: Prototyp 1 (bez zasilania bateryjnego i GPS wewnętrznego)



Oprogramowanie

Oprogramowanie wbudowane jest napisane w języku C++ i oparte o ekosystem Arduino. Korzysta ono ekstensywnie z funkcji systemu freeRTOS pozwalających na jednoczesne wykonywanie wielu zadań. Program pozwala na konfigurację zachowania urządzenia dzięki wczytywaniu pliku *settings.json* z głównego katalogu karty SD. Poniżej umieszczono wzorzec zawartości pliku:

```
{
  "ssid": "nazwa sieci, do której podłączyć urządzenie",
  "password": "hasło sieci, do której połączyć urządzenie",
  "apSsid": "nazwa sieci generowanej przez urządzenie",
  "apPassword": "hasło sieci generowanej przez urządzenie",
  "domain": "nazwa domeny urządzenia we własnej sieci",
  "otaHostname": "nazwa do programowania zdalnego",
  "otaPassword": "hasło do programowania zdalnego",
  "beta": "współczynnik zbiegania filtra IMU"
}
```

Oprogramowanie dokonuje zebrania danych ze wszystkich dostępnych sensorów, a następnie stosuje filtry oraz generuje trendy na podstawie uśrednień kroczących. Klasą zajmującą się dokonywaniem opracowań danych jest klasa CourseFilter.

Oprogramowanie umożliwia także uaktualnianie urządzenia bez dostępu do portu USB mikrokontrolera dzięki połączeniu WiFi. Pozwala to uniknąć rozbierania prototypu przy każdej poprawce oprogramowania. Klasą umożliwiającą aktualizację bezprzewodową jest klasa OTA.

Pozostałe funkcje oprogramowania zostały opisane w modułach, z którymi są powiązane.

Załączniki

1. Załącznik „Dokumentacja oprogramowanie wbudowane.pdf”

Załącznik jest dokumentacją oprogramowania wbudowanego wygenerowaną na podstawie dokumentacji wewnętrznej umieszczonej w kodzie źródłowym. Do wygenerowania dokumentacji w formie dokumentu użyto narzędzia doxygen z wyjściem LaTeX. Dokumentacja jest napisana w formatowaniu javadoc oraz języku angielskim, co jest normą branżową stosowaną przy pisaniu dokumentacji inline (język dokumentacji jest dopasowany do języka kodu, czyli języka angielskiego). Dokument przygotowano z uwzględnieniem zasad tworzenia dokumentów dostępnych.